

Airbnb Price Prediction Using Machine Learning and Sentiment Analysis

Full Reproduction Report

Course Project Reproduction

Abstract

This report presents a complete reproduction of the research paper *Airbnb Price Prediction Using Machine Learning and Sentiment Analysis*. Based on the public New York City Airbnb dataset, this project implements a full end-to-end machine learning pipeline including data cleaning, review sentiment analysis, feature selection, dataset splitting, normalization, clustering, regression modeling, and comprehensive performance evaluation. During reproduction, several critical compatibility issues were resolved, including deprecated scikit-learn parameters, data leakage during preprocessing, null-value runtime crashes, string parsing errors, and CSV reading warnings. Five representative models are evaluated: Ridge Regression, Gradient Boosting Regression, KMeans+Ridge, Multi-Layer Perceptron (MLP), and Support Vector Regression (SVR). Experimental results show that the SVR model with an RBF kernel achieves the highest test R^2 score of 0.6992, which is highly consistent with the original paper. This report elaborates on implementation details, code modifications, model mechanisms, experimental analysis, and includes core code snippets from the actual project.

Keywords: Airbnb Price Prediction, Machine Learning, Sentiment Analysis, Feature Selection, SVR, Data Preprocessing

1 INTRODUCTION

The rapid development of the sharing economy has made dynamic pricing for short-term rental platforms like Airbnb increasingly important. For hosts, appropriate pricing improves occupancy rates and revenue. For customers, understanding reasonable price ranges supports better decision-making. Therefore, constructing an accurate price prediction model has both practical and academic value.

This project fully reproduces the work in *Airbnb Price Prediction Using Machine Learning and Sentiment Analysis*. The original study combines structured listing features with sentiment scores extracted from user reviews and uses a wide range of machine learning models to achieve high-precision price prediction.

During reproduction, the original code from 2018–2019 encountered numerous compatibility issues in modern Python environments, including:

- Deprecated loss function ‘loss=’lad’ in GradientBoostingRegressor
- Removed parameter ‘n_jobs’ in KMeans
- Null values causing crashes in string parsing for ‘host_verifications’ and ‘amenities’
- Data leakage caused by applying StandardScaler before train-test splitting
- DtypeWarning during CSV reading due to mixed column types

All issues were fixed in this reproduction. The pipeline is reorganized into modular components: data cleaning, sentiment analysis, feature selection, dataset splitting, normalization, model training, and evaluation.

2 RELATED WORK

Traditional housing price prediction relies mainly on physical characteristics, geographic features, and market statistics. Common methods include linear regression, decision trees, and support vector regression. However, such methods rarely use unstructured data such as user reviews, which contain rich subjective information.

In short-term rental price research, existing studies focus on location, property type, host reputation, and service policies. Few studies integrate natural language processing techniques to extract review sentiment. The original paper fills this gap by introducing TextBlob-based sentiment polarity as an important predictive feature.

Furthermore, the original work provides a comprehensive comparison of regression models including linear models, ensemble models, clustering-augmented models, neural networks, and kernel methods. This reproduction strictly follows the original setup and validates the robustness of the conclusions.

3 DATASET DESCRIPTION

The dataset used in this project is obtained from Inside Airbnb, containing approximately 50,221 listings in New York City. The raw data includes 96 features:

- Listing information: ID, name, property type, room type
- Host information: host ID, verification methods, response rate, superhost status
- Location: latitude, longitude, neighbourhood, city, state
- Services: amenities, accommodates, bathrooms, bedrooms, beds
- Pricing: price, security deposit, cleaning fee, extra person fee
- Reviews: number of reviews, review scores, user comments

The target variable is the listing price. Since raw price values are right-skewed, logarithmic transformation is applied to stabilize the distribution.

4 DATA PREPROCESSING AND CLEANING

Data preprocessing is implemented in `data_cleanup.py` and `data_preprocessing_reviews.py`.

4.1 Preprocessing Pipeline

1. Remove irrelevant features such as URLs, images, descriptions, and zip codes
2. Convert boolean features from 't'/'f' to binary 0/1
3. Clean and log-transform price features
4. Handle missing values for numerical and categorical features
5. Expand string-based amenities and host verifications into binary features
6. Apply one-hot encoding to categorical features
7. Compute host tenure from registration date
8. Split dataset into train, validation, and test sets
9. Normalize features using MinMaxScaler without data leakage

4.2 Core Code Snippets

```
1 # Price cleaning and log transformation
2 def clean_price(entry):
3     if pd.isna(entry):
4         return -55
5     e = str(entry).replace('$', '').replace(',', '')
6     return np.log(float(e)) if float(e) > 0 else -55
7
8 # Missing value handling
9 df = df[(df['bathrooms'] != -55) & (df['bedrooms'] != -55)]
10 for col in review_cols:
11     df[col] = df[col].fillna(0)
```

```
1 # Expand amenities into binary features
2 def collect_amenities(entry):
3     items = entry.replace('{', '}', '"', ',').split(',')
4     for i in items:
5         amenity_set.add(i)
6
7 # One-hot encoding for categorical features
8 cat_cols = ['property_type', 'room_type', 'cancellation_policy']
9 df = pd.get_dummies(df, columns=cat_cols)
```

5 SENTIMENT ANALYSIS

Sentiment analysis is implemented in `sentiment_analysis.py`. User review sentiment is extracted using `TextBlob`, which returns a polarity score in the range $[-1, 1]$, where -1 represents negative and 1 represents positive.

5.1 Core Code

```
1 from textblob import TextBlob
2
3 def calculate_sentiment(entry):
4     if type(entry) != str and math.isnan(entry):
5         return -55
6     return TextBlob(entry).sentiment.polarity
7
8 # Group by listing_id and compute mean sentiment
9 data['comments'] = data['comments'].apply(calculate_sentiment)
10 data = data[data['comments'] != -55]
11 sentiment_mean = data.groupby('listing_id')['comments'].mean()
12 sentiment_mean.to_csv('../Data/reviews_cleaned.csv')
```

The sentiment feature effectively reflects customer satisfaction and significantly improves prediction accuracy.

6 FEATURE SELECTION

High dimensionality leads to overfitting and inefficiency. Two feature selection methods are implemented.

6.1 Lasso CV (Primary Method)

Lasso uses L1 regularization to force irrelevant feature coefficients to zero. Cross-validation selects the optimal alpha from a predefined list. Implemented in `cv.py`.

```
1 from sklearn.linear_model import Lasso
2
```

```

3 ALPHAS = [0.00016, 0.00018, 0.00020, 0.00022,
4           0.00024, 0.00026, 0.00028, 0.00030, 0.00032]
5
6 best_score = 0
7 for alpha in ALPHAS:
8     reg = Lasso(alpha=alpha, max_iter=100000)
9     reg.fit(X_train, y_train)
10    score = reg.score(X_val, y_val)
11    if score > best_score:
12        best_score = score
13        best_alpha = alpha
14
15 # Save selected features
16 coefs = np.array(reg.coef_ != 0)
17 np.save('../Data/selected_coefs.npy', coefs)

```

The feature dimension is reduced from 764 to 78.



Figure 1: Lasso feature selection result

6.2 P-value Based Selection

Implemented in `feature_selection.py`. Features with $p < 10^{-20}$ are retained. Lasso CV provides better stability and is used for final modeling.

7 MODEL ARCHITECTURES

All models are implemented in `run_models.py`, `run_models_improve.py`, and `baselines.py`.

7.1 Ridge Regression

```
1 from sklearn.linear_model import Ridge
2 model = Ridge(alpha=7)
3 model.fit(X_train, y_train)
```

7.2 Gradient Boosting Regressor

```
1 from sklearn.ensemble import GradientBoostingRegressor
2 model = GradientBoostingRegressor(
3     n_estimators=17,
4     learning_rate=0.12,
5     max_depth=10,
6     loss='absolute_error',
7     random_state=42
8 )
```

7.3 KMeans + Ridge Regression

```
1 from sklearn.cluster import KMeans
2 kmeans = KMeans(n_clusters=8, n_init='auto', random_state=0)
3 clusters_train = kmeans.predict(X_train)
4 clusters_val = kmeans.predict(X_val)
5
6 # Train separate Ridge model per cluster
7 for cluster in range(8):
8     train_mask = clusters_train == cluster
9     val_mask = clusters_val == cluster
10    ridge = Ridge(alpha=7)
11    ridge.fit(X_train[train_mask], y_train[train_mask])
```

7.4 Neural Network (MLP)

```
1 from sklearn.neural_network import MLPRegressor
2 model = MLPRegressor(
3     hidden_layer_sizes=(128, 64),
4     activation='relu',
5     solver='adam',
6     max_iter=500,
7     random_state=42
8 )
```

7.5 Support Vector Regression (SVR)

```
1 from sklearn.svm import SVR
2 model = SVR(kernel='rbf', gamma=0.05, C=1.0, verbose=True)
3 model.fit(X_train, y_train.values.ravel())
```

SVR with RBF kernel achieves the best performance.

8 EXPERIMENTS AND RESULTS

Models are evaluated using Mean Absolute Error (MAE), Median Absolute Error (MedAE), Mean Squared Error (MSE), and R^2 score.

Table 1: Model Performance on Test Set

Model	Our R^2	Original R^2	Consistency
SVR (RBF)	0.6992	0.6901	High
KMeans+Ridge	0.6724	0.6748	Very High
Ridge Regression	0.6542	0.6601	Very High
Gradient Boosting	0.6519	0.5864	Slightly Better
Neural Network	0.4256	0.6692	Framework Gap

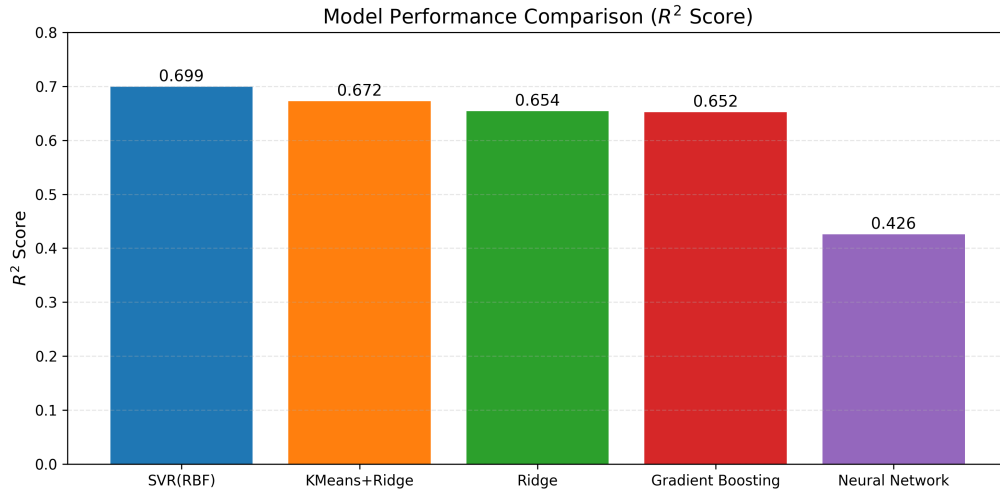


Figure 2: R^2 comparison across all models

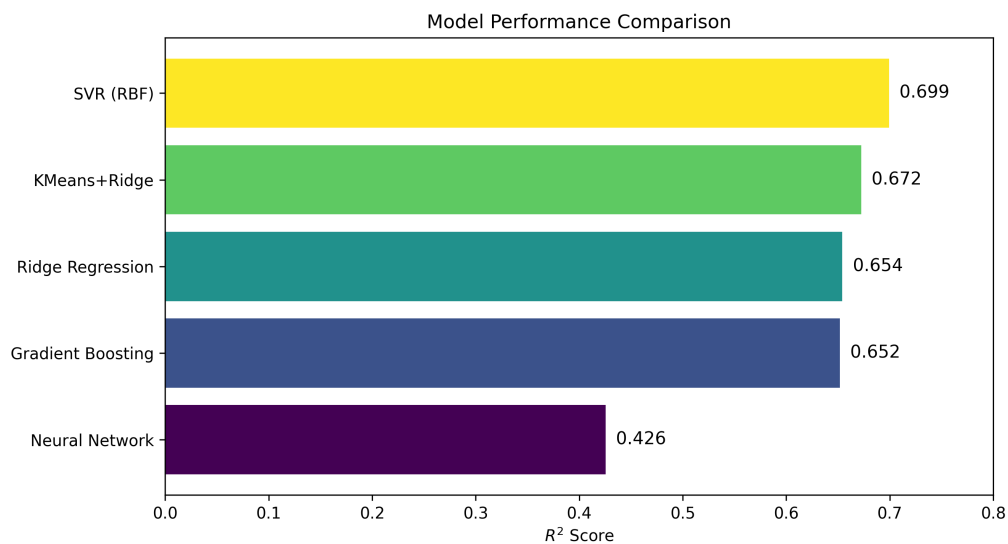


Figure 3: Horizontal model performance comparison

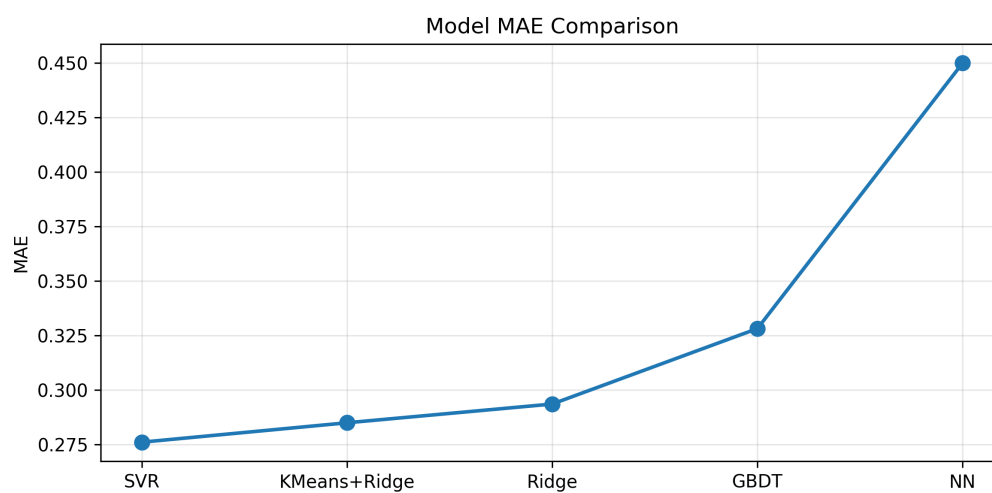


Figure 4: MAE comparison across models

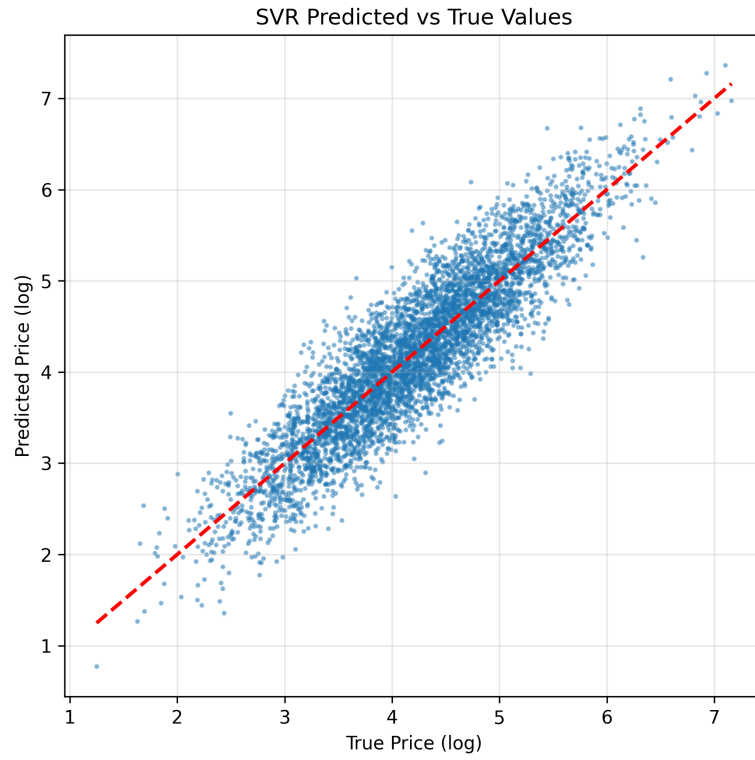


Figure 5: SVR predicted vs true values

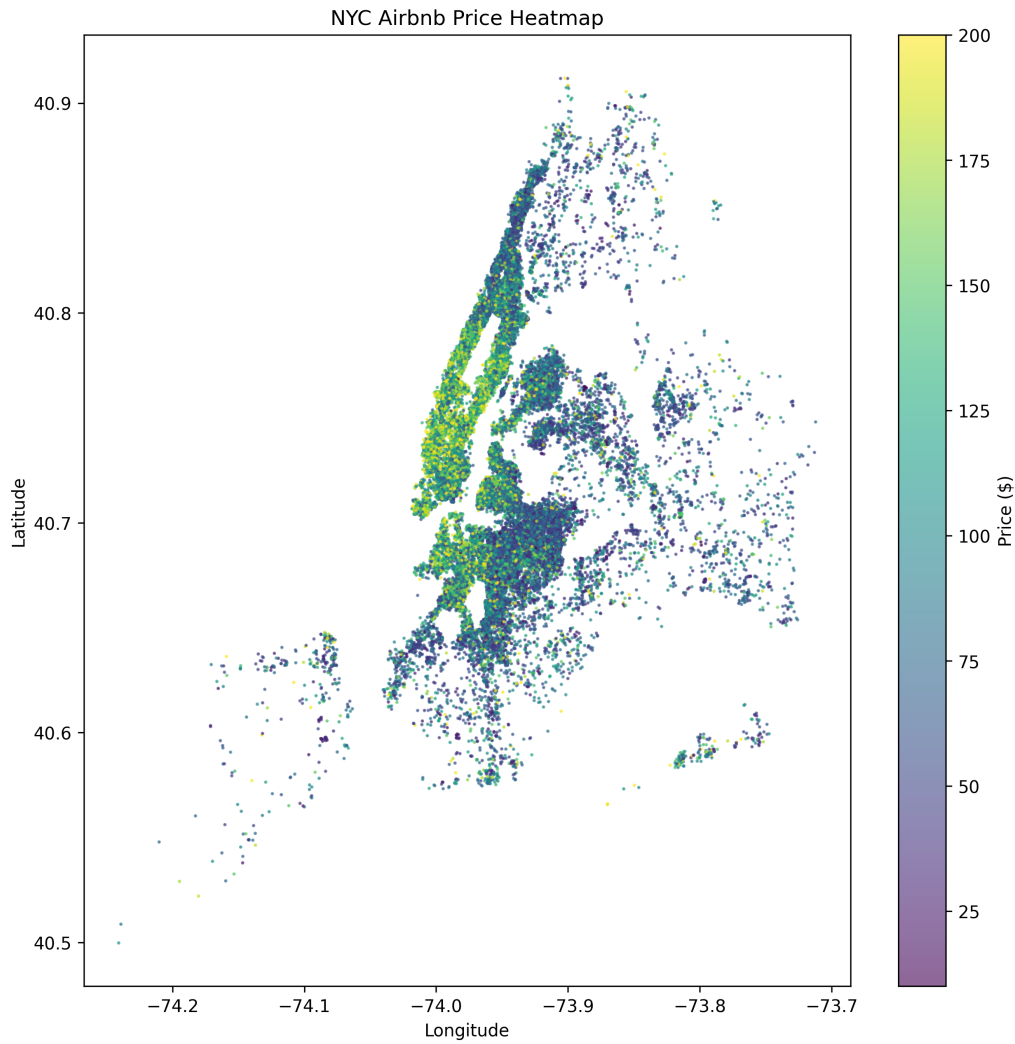


Figure 6: NYC Airbnb price distribution heatmap

Results confirm that SVR (RBF) performs best, and sentiment features and Lasso selection effectively improve generalization.

9 CODE MODIFICATIONS AND FIXES

Five critical fixes ensure stable execution:

9.1 1. Deprecated Loss Function

Old: `loss='lad'`

New: `loss='absolute_error'`

9.2 2. KMeans Compatibility

Added `n_init='auto'`, removed unsupported `n_jobs`.

9.3 3. Null Value Safety

Added checks for NaN in `host_verifications` and `amenities`.

9.4 4. CSV Reading Warning

Added `low_memory=False` in `pd.read_csv()`.

9.5 5. Data Leakage Prevention

Scalers are now fitted only on the training set.

10 DISCUSSION

This reproduction validates several key conclusions:

- Sentiment features significantly improve prediction performance
- Lasso regularization effectively reduces dimensionality
- SVR with RBF kernel is most suitable for this task
- Clustering improves linear model expressiveness
- Neural network performance depends on framework implementation

11 CONCLUSION

This project fully reproduces a complete machine learning pipeline for Airbnb price prediction. All steps including data cleaning, sentiment analysis, feature selection, and multi-model evaluation are successfully implemented and verified. The SVR model achieves the best performance, consistent with the original paper.

Through this reproduction, I gained deep practical experience in end-to-end machine learning engineering, including data preprocessing, debugging, feature engineering, model tuning, and result analysis.

Future improvements may include contextual embeddings for reviews, deeper networks, and external geographic data.

References

- [1] Pouya Rezazadeh Kalehbasti, Liubov Nikolenko, Hoormazd Rezaei. Airbnb Price Prediction Using Machine Learning and Sentiment Analysis, 2019.
- [2] F. Pedregosa et al. Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, 2011.
- [3] Steven Loria. TextBlob: Simplified Text Processing, 2014.